

Análisis Exhaustivo y Guía de Implementación del Proyecto Biomédico RuView: De la Teoría Electromagnética a la Telemetría Clínica Perimetral

Introducción al Paradigma de la Inteligencia Ambiental y Detección por Radiofrecuencia

La intersección entre la ingeniería de telecomunicaciones y la instrumentación biomédica ha gestado una revolución en las metodologías de monitorización no invasiva. Tradicionalmente, la telemetría clínica y la inteligencia espacial en entornos cerrados han dependido de dos vectores tecnológicos principales: la visión artificial (cámaras ópticas e infrarrojas) y los dispositivos portátiles (wearables). Ambos paradigmas presentan vulnerabilidades sistémicas inherentes. Los sistemas ópticos comprometen gravemente la privacidad anatómica, sufren de oclusiones físicas y su rendimiento se degrada en condiciones de baja luminosidad o presencia de material particulado (humo, polvo). Por otro lado, los dispositivos portátiles requieren el cumplimiento continuo del usuario (user compliance), mantenimiento de baterías y contacto directo con la piel, lo cual es subóptimo en unidades de cuidados intensivos, residencias geriátricas o en la monitorización prolongada del sueño.

El proyecto RuView (también referenciado bajo la nomenclatura técnica WiFi DensePose) surge como una respuesta a estas limitaciones, estableciendo un ecosistema de hardware y software fundamentado en la "privacidad por física". RuView convierte señales de radiofrecuencia (RF) de redes WiFi comerciales y radares de ondas milimétricas (mmWave) en un tejido de detección espacial en tiempo real, operando sin un solo píxel de vídeo y eliminando la necesidad de que el sujeto porte dispositivos electrónicos.

La propuesta arquitectónica del repositorio evalúa y extrae parámetros complejos como la presencia humana, el recuento de ocupación, la respiración, la frecuencia cardíaca, la variabilidad de la frecuencia cardíaca (HRV), la estimación de la presión arterial, la clasificación del sueño y la estimación de la pose humana (esqueleto de 17 puntos clave). La capacidad fundamental de las ondas electromagnéticas en las bandas de 2.4 GHz, 5 GHz y 60 GHz para penetrar materiales dieléctricos opacos permite a este sistema operar a través de paredes de yeso o concreto, detectando individuos en habitaciones contiguas. Aunque el repositorio indica que la plataforma se encuentra en fase beta y está orientada principalmente a la investigación, creación de prototipos y computación de borde (Edge AI), su grado de madurez analítica presenta un modelo fundacional para el futuro de los hospitales inteligentes y el envejecimiento asistido.

Este informe técnico disecciona la arquitectura matemática, fisiológica y de software del ecosistema RuView, culminando con un plan de despliegue exhaustivo para su implementación en ordenadores portátiles de grado consumidor y su integración nativa con infraestructuras de almacenamiento en la nube, específicamente Google Drive, para el entrenamiento continuo de

modelos biomédicos.

Fundamentos Físicos de la Interacción Electromagnética Humana

Para comprender la viabilidad clínica del sistema, es imperativo establecer cómo las ondas electromagnéticas interactúan con la fisiología humana y cómo el hardware extrae métricas de estas interacciones. La plataforma utiliza principalmente dos modalidades de detección de radiofrecuencia.

Información del Estado del Canal (CSI) en Redes WiFi

Las infraestructuras de red inalámbrica modernas (estándares IEEE 802.11n, ac y ax) emplean la Multiplexación por División de Frecuencias Ortogonales (OFDM). En este esquema, el espectro de transmisión se divide en múltiples subportadoras ortogonales. La Información del Estado del Canal (CSI) es una matriz de datos complejos que las tarjetas de red calculan para ecualizar la señal y mitigar la distorsión del canal inalámbrico.

Físicamente, cuando un router transmite un paquete a un nodo receptor (por ejemplo, un microcontrolador ESP32-S3), la onda no viaja únicamente en línea recta. Se refleja en las paredes, se difracta en los bordes de los muebles y se dispersa al impactar contra el cuerpo humano, que está compuesto en un 60-70% por agua y actúa como un poderoso reflector y atenuador dieléctrico.

La ecuación fundamental que modela este fenómeno en el dominio de la frecuencia es $Y = H \cdot X + N$, donde X es el vector de la señal transmitida, Y es la señal recibida, N representa el ruido térmico y ambiental, y H es la matriz CSI. Cada elemento de la matriz H contiene una amplitud y una fase que describen la atenuación y el desplazamiento de fase sufridos por cada subportadora.

A medida que una persona respira, el leve movimiento mecánico de expansión y contracción de su caja torácica altera microscópicamente la longitud de las trayectorias multitrayecto (multi-path) de la señal. Al muestrear la matriz CSI a una frecuencia de 20 Hz con nodos de detección de bajo costo como el ESP32-S3 (configurado en una malla multiestática), RuView captura estas variaciones. El procesamiento de señales digitales limpia el ruido de fase del oscilador local y aplica filtros para extraer las huellas del movimiento humano de la topología estática de la habitación.

Radar FMCW de Ondas Milimétricas (Banda de 60 GHz)

Mientras que el WiFi CSI proporciona resolución a nivel de habitación y permite la detección de presencia a través de las paredes, la estimación de constantes vitales de grado clínico requiere una resolución espacial significativamente mayor. Para este propósito, RuView integra radares de onda continua de frecuencia modulada (FMCW) en la banda de 60 GHz, como el módulo Sseed MR60BHA2, que se acopla a un microcontrolador ESP32-C6.

Un radar FMCW emite pulsos continuos o "chirps" cuya frecuencia aumenta linealmente a lo largo del tiempo. Cuando esta señal impacta contra un sujeto estacionario, se refleja de vuelta al radar. El receptor mezcla la señal de transmisión original con la señal reflejada retrasada para generar una señal de frecuencia intermedia (IF). La frecuencia de esta señal IF es directamente proporcional a la distancia del objetivo.

Sin embargo, el avance crucial para aplicaciones biomédicas reside en el análisis de fase de la señal IF. A 60 GHz, la longitud de onda (λ) es de apenas 5 milímetros. Cualquier desplazamiento de la pared torácica modula la fase de la señal IF generada. La relación matemática establece que el desplazamiento Δd genera un desplazamiento de fase $\Delta \phi = \frac{4\pi}{\lambda} \Delta d$. Debido a la minúscula longitud de onda, el radar es capaz de resolver desplazamientos físicos fraccionales por debajo del milímetro.

Fenómeno Fisiológico	Desplazamiento Mecánico Torácico	Frecuencia Fisiológica Típica
Ciclo Respiratorio	$0.1 \text{ a } 1.0 \text{ mm}$	12 a 25 respiraciones por minuto
Pulso Cardíaco (Balistocardiografía)	$0.01 \text{ a } 0.1 \text{ mm}$	60 a 100 latidos por minuto

Como se observa en los rangos descritos en la documentación del módulo médico, el radar FMCW acoplado al ESP32-C6 aísla estas vibraciones micrométricas del tórax con un coste de hardware no superior a los \$15 USD.

Telemetría Biomédica Avanzada y Fusión de Sensores

El repositorio detalla una arquitectura orientada a extraer conclusiones clínicas a partir de las señales de radiofrecuencia en bruto. El núcleo de esta funcionalidad médica se encuentra en los scripts `bp_estimator.py` y `vitals_suite.py` dentro del directorio `examples/medical/`.

Aislamiento de Señales Fisiológicas mediante Filtrado y Grafos de Coherencia

Cuando una persona está sentada frente al sensor, la señal bruta contiene una mezcla de la expansión pulmonar, la percusión miocárdica contra la pared torácica, y el ruido ambiental. El motor de procesamiento de RuView (integrado a través del marco analítico RuVector) aborda esta separación utilizando un análisis de corte mínimo dinámico (dynamic min-cut analysis), tratando las señales fisiológicas como un grafo de coherencia. Este enfoque matemático separa automáticamente los artefactos de movimiento macroscópicos del usuario y las interferencias de la señal vital pura.

Posteriormente, el sistema aplica filtros de paso banda (Bandpass filters) para aislar las métricas específicas :

1. **Frecuencia Respiratoria (BR):** Extraída al aplicar un filtro de paso banda estricto entre 0.1 y 0.5 Hz sobre la varianza de fase envuelta, lo que corresponde a un rango lógico de 6 a 30 respiraciones por minuto.
2. **Frecuencia Cardíaca (HR):** Procesada filtrando la banda entre 0.8 y 2.0 Hz , que correlaciona con la ventana de 48 a 120 latidos por minuto. El algoritmo emplea la detección de cruce por cero (zero-crossing) para cuantificar los latidos y generar una serie temporal precisa.

Estimación de Presión Arterial sin Contacto (HRV-BP)

La característica clínica más innovadora documentada en el repositorio es la capacidad de estimar la presión arterial de forma continua sin utilizar brazaletes o manguitos presurométricos. El módulo `bp_estimator.py` infiere las presiones sistólica y diastólica aprovechando la profunda relación fisiológica entre la Variabilidad de la Frecuencia Cardíaca (HRV) y el tono cardiovascular.

El sistema nervioso autónomo regula tanto el gasto cardíaco como la resistencia vascular periférica, los dos determinantes de la presión arterial. El estimador funciona ejecutando un proceso algorítmico secuencial :

1. Extrae los intervalos latido a latido continuos (intervalos R-R equivalentes) de la serie temporal del radar.
2. Computa métricas estadísticas y espectrales de la HRV, predominantemente la Desviación Estándar de los Intervalos Normal a Normal (SDNN) y la proporción de baja frecuencia sobre alta frecuencia (cociente LF/HF).
3. Aplica modelos de correlación fisiológica:
 - Una frecuencia cardíaca más alta indica una activación del sistema nervioso simpático, correlacionándose con una mayor presión arterial.
 - Un SDNN reducido (pérdida de variabilidad parasimpática global) denota rigidez y alto tono vascular, traducándose en una presión arterial más alta.
 - Un cociente LF/HF elevado evidencia dominancia simpática sobre la parasimpática, lo que sugiere hipertensión transitoria.

Las estimaciones generadas se clasifican y muestran en el panel de control basándose en las directrices clínicas de la Asociación Americana del Corazón (AHA) :

Clasificación AHA	Presión Sistólica (mmHg)	Presión Diastólica (mmHg)
Normal	< 120	< 80
Elevada	120 - 129	< 80
Hipertensión Nivel 1	130 - 139	80 - 89
Hipertensión Nivel 2	≥ 140	≥ 90

Para alcanzar una precisión clínica funcional (errores de $\pm 8-12$ mmHg), el sistema requiere que el sujeto permanezca estacionario y se introduzca una calibración inicial. El script de inferencia admite argumentos de entrada de un esfigmomanómetro de referencia para ajustar la base tensional del individuo (`--cal-systolic 120 --cal-diastolic 80 --cal-hr 72`). Sin calibración, el sistema mantiene una precisión de $\pm 15-20$ mmHg, lo cual es suficiente para rastrear derivas tensionales relativas provocadas por estrés agudo.

Monitorización Autónoma del Sueño y Análisis de Estrés

El paquete `vitals_suite.py` extrapola el análisis de ondas continuas utilizando la estadística de Welford para generar líneas base dinámicas en tiempo real. En entornos experimentales, esto ha permitido a los desarrolladores detectar respuestas fisiológicas anómalas: caídas de frecuencia cardíaca de múltiples desviaciones estándar durante ejercicios de relajación o picos de frecuencia respiratoria de 2.2 desviaciones estándar sobre el umbral individual. Esta sensibilidad es el pilar para los algoritmos de estadiaje del sueño. Dado que la arquitectura del sueño está íntimamente ligada a la modulación del sistema nervioso autónomo, el sistema determina la etapa del sueño sin encefalogramas (EEG):

- **Sueño Profundo (NREM N3):** Identificado por frecuencia respiratoria baja, frecuencia cardíaca estable baja y mínima variabilidad HRV.
- **Sueño REM:** Caracterizado por frecuencia cardíaca moderada acompañada de ráfagas irregulares y alta variabilidad cardiorrespiratoria.
- **Sueño Ligero (NREM N1/N2):** Parámetros transicionales de variabilidad moderada.

Además, el algoritmo incluye métricas de meditación, computando una "puntuación de relajación profunda" cuando detecta la triada fisiológica: caída del coeficiente de variación respiratoria (respiración altamente regular), ralentización del ritmo miocárdico basal y un incremento agudo en la HRV mediado por el nervio vago.

Arquitectura de Software y Diseño Guiado por el Dominio (DDD)

Soportar una ingesta masiva de matrices matemáticas desde sensores perimetrales asíncronos y mantener baja latencia inferencial requiere una disciplina arquitectónica rigurosa. RuView está estructurado predominantemente en el lenguaje Rust dentro de un espacio de trabajo multifase (Workspace v2/).

El directorio `docs/ddd/README.md` expone la utilización estricta del Diseño Guiado por el Dominio (Domain-Driven Design, DDD). El ecosistema no organiza su código por capas técnicas (controlador, vista, base de datos), sino alrededor de los dominios específicos de problemas espaciales y electromagnéticos.

Modelos y Contextos Delimitados (Bounded Contexts)

La base de código aísla el conocimiento en tres modelos fundamentales de negocio y procesamiento de hardware :

1. **rvCSI (Edge RF Sensing Runtime):** Es el corazón del procesamiento en el borde. Gestiona la ingesta de múltiples fuentes de CSI, su validación criptográfica y de integridad de tramas, la normalización temporal y la extracción de eventos. Está fragmentado en 7 contextos delimitados: *Capture*, *Validation*, *Signal*, *Calibration*, *Event*, *Memory*, y *Agent*.
2. **WiFi-Mat:** Un modelo especializado en respuestas ante desastres estructurales y evaluación de víctimas en masa.
3. **CHCI (Coherent Human Channel Imaging):** Un modelo avanzado para la reconstrucción submilimétrica de la superficie del cuerpo humano y el mapeo espacial denso.

Los servicios de dominio son los motores que transforman las señales brutas en observaciones

semánticas :

- FrameValidationService: Aplica políticas matemáticas estrictas. Rechaza tramas no finitas, corrupciones de datos en el canal UDP o errores de monotonidad temporal. Inyecta calidad computacional a las tramas aceptadas.
- SignalProcessingService: Agrupa los paquetes validados en ventanas temporales (CsiWindow) y ejecuta la Transformada Rápida de Fourier (FFT), calculando umbrales de energía de movimiento.
- BaselineDeltaService y CalibrationService: Los sistemas RF requieren conocer cómo se ve la habitación vacía. Estos servicios aprenden un perfil estable, discriminando los muebles y reflexiones estáticas de la firma humana móvil.
- EventDetectionService: Mantiene máquinas de estado por individuo/objeto para inyectar semántica (por ejemplo, detectar la transición de de pie a una caída súbita).
- RfMemoryService y EmbeddingService: Producen incrustaciones temporales vectoriales para búsquedas de similitud (análisis de escenarios previos recordados).

Backbone Neuronal, Codificadores Contrastivos y el Ecosistema Hugging Face

Las heurísticas tradicionales de procesamiento de señales son propensas a errores ante cambios ambientales drásticos. Para mitigar esto y lograr la estimación de la pose humana (DensePose), RuView emplea el aprendizaje profundo autosupervisado, integrando la plataforma Hugging Face en su canalización de inferencia principal.

El modelo base, etiquetado como ruvnet/wifi-densepose-pretrained, logra una hazaña notable: transformar los rebotes ciegos del WiFi en esqueletos articulares humanos de 17 puntos clave estandarizados bajo la arquitectura COCO.

Preentrenamiento y Transferencia de Supervisión

El método para enseñar a una red neuronal a "ver" huesos humanos usando radiofrecuencia se denomina destilación de conocimiento de modalidad cruzada (Cross-modal knowledge distillation). En el entorno de preentrenamiento documentado :

1. Se empareja una cámara óptica temporal con un hardware ESP32-S3 en una habitación.
2. La cámara ejecuta un modelo monocular de estimación de profundidad (MiDaS) pesado para generar nubes de puntos 3D de alta resolución (19,200+ puntos de profundidad por fotograma).
3. Simultáneamente, la antena WiFi captura la deformación de la matriz CSI.
4. La red neuronal auto-supervisada es entrenada para predecir la pose computada por la cámara *utilizando exclusivamente la matriz CSI* como dato de entrada.
5. Tras 12.2 millones de pasos de entrenamiento sobre 60,630 fotogramas recolectados durante 8 horas de monitorización, la red adquiere una correlación cruzada robusta. En ese punto, la cámara óptica se desconecta y se retira de la arquitectura.

Inferencia en el Borde y Contenedores RVF

El sistema prescinde completamente de Python en el servidor de producción para evitar la fragmentación y el alto consumo de memoria, utilizando el ecosistema Rust (con aceleración Candle para tensores) en ordenadores o adaptando los pesos a microcontroladores de baja potencia utilizando WebAssembly (WASM). El codificador generativo contrastivo se comprime en un espacio dimensional de 128 parámetros. Para microprocesadores de grado comercial extremo (IoT perimetral), el sistema aplica una cuantificación paramétrica de 4 bits, permitiendo que el motor de red neuronal artificial encaje en apenas 8 Kilobytes de memoria SRAM estática.

La distribución y carga fluida de estos parámetros neuronales se orquesta a través del formato de contenedores **RVF** (RuVector Format). Un archivo RVF consolida los pesos de la red neuronal, los índices precomputados de Búsqueda Jerárquica Navegable de Mundo Pequeño (HNSW) para la recuperación eficiente, los diccionarios de cuantificación y las rutinas binarias WASM. El punto final de la API (POST /api/v1/models/load) carga dinámicamente este contenedor de una sola pasada en el servidor en caliente sin reinicios.

Teoría del Caos Espacial y Clasificadores Adaptativos en Entornos Dinámicos

La adopción de esquemas de inteligencia en el extremo final de la red plantea problemas de eficiencia y falsos positivos ante cambios ambientales no biológicos (corrientes térmicas, ventiladores mecánicos). La documentación de adaptive-learning.md expone un paradigma brillante que descarta temporalmente las redes neuronales pesadas a favor del uso de la dinámica de sistemas no lineales y la teoría del caos.

Para determinar si una disrupción electromagnética en la sala corresponde a un evento humano válido o a un ruido mecánico, el microcontrolador rastrea el vector del canal y computa el máximo exponente de Lyapunov de la serie temporal. El comportamiento del ecosistema espacial es etiquetado matemáticamente:

1. **Atractor Puntual (Point Attractor):** $\lambda < -0.01$. La señal converge inexorablemente a un punto estable. Representa un estado termodinámico y estructural estacionario. No hay agentes biológicos ni mecánicos moviéndose.
2. **Ciclo Límite (Limit Cycle):** $\lambda \approx 0$. La perturbación describe una órbita cíclica determinista. Correlaciona directamente con firmas de equipos HVAC (aire acondicionado), ventiladores oscilantes o motores. La interferencia RF tiene un período predecible.
3. **Atractor Extraño (Strange Attractor):** $\lambda > 0.01$. El comportamiento característico del caos determinista acotado. Indica movimiento biológico complejo: un humano caminando, gesticulando o respirando.

Una métrica clave detectada es la llamada "Basin Departure" (Salida de la Cuenca). Si la métrica de variación excede en un factor superior a 3 veces el radio del atractor convergente, el sistema infiere un evento radical de alteración ambiental (entrada de una nueva persona, apertura forzada de una puerta, o introducción de nuevo mobiliario masivo).

El Optimizador Meta Adaptativo

El motor integrado en Rust (Irn_meta_adapt.rs) emplea un optimizador iterativo de escalada de colinas (hill-climbing optimizer) para purgar falsos positivos a través del auto-ajuste de 8 variables en tiempo real. Utilizando la retroalimentación ambiental continua, el algoritmo altera de manera autónoma umbrales que dictan las condiciones probabilísticas de movimiento y presencia sin intervención del administrador.

Parámetro Umbral Adaptativo	Valor Predeterminado Inicial	Rango Tolerado de Búsqueda
Sensibilidad de Presencia	0.05	0.01 - 0.50
Sensibilidad de Movimiento	0.10	0.02 - 1.00
Tolerancia de Coherencia de Grafo	0.70	0.30 - 0.99

Transición Sectorial: Aplicaciones Industriales y Triage en Desastres

Aunque concebido con una potente arquitectura biomédica personal, el módulo WiFi-Mat y las rutinas perimetrales de seguridad expanden el uso del repositorio más allá del ámbito de la atención domiciliaria, estableciendo directrices rigurosas para operaciones industriales y de rescate masivo.

Monitorización de Espacios Confinados y Seguridad en Planta

El módulo industrial aborda fallos fatales tradicionales en entornos de factoría donde el humo, gases tóxicos o recintos metálicos profundos imposibilitan el uso de cámaras de supervisión de CCTV. El repositorio sugiere un despliegue topológico con al menos dos nodos ESP32. Un nodo supervisor ("Sensor A") instalado en la escotilla principal de entrada/salida y un "Sensor B" en el interior del recinto (si la seguridad intrínseca lo permite).

El flujo de procesamiento perimetral exporta identificadores numéricos de eventos a una red local mediante paquetes UDP o integraciones con Controladores Lógicos Programables (PLCs) existentes a través de gateways Modbus TCP. El catálogo de eventos para espacios confinados y proximidad de vehículos se define bajo estructuras de código rígidas:

Rango de ID	Módulo Categorizado	Nombre del Evento y Naturaleza Semántica
500 - 502	Proximidad de Carretilla	PROXIMITY_WARNING, VEHICLE_DETECTED, HUMAN_NEAR_VEHICLE
510 - 514	Seguridad en Espacios Confinados	WORKER_ENTRY, WORKER_EXIT, BREATHING_OK

Esta integración se traduce en que, si un operario sucumbe a emanaciones de monóxido de carbono dentro de un depósito ciego y pierde el conocimiento (cesa el movimiento macroscópico), la placa detectará un cambio en el patrón vital. Si la señal BREATHING_OK desaparece o disminuye rápidamente bajo el umbral de bradipnea sin que un evento secuencial de WORKER_EXIT se registre previamente en el Sensor A, el sistema puede disparar una alarma de circuito de seguridad por fallo humano en fracciones de segundo.

Sistema de Respuesta ante Desastres (START Triage)

Un componente crítico adicional del modelo Bounded Context WiFi-Mat es la arquitectura de soporte en estructuras colapsadas. Las ondas de frecuencias inferiores en 2.4 GHz tienen formidables propiedades de penetración de pared y escombros. El sistema localiza espacialmente supervivientes inmovilizados, rastrea la respiración irregular causada por trauma contuso y clasifica la severidad fisiológica del daño utilizando protocolos de triaje masivo estandarizados (Simple Triage and Rapid Treatment - START), guiando de forma remota a los operadores de los equipos de rescate con información sobre el número de personas con vida y sus posiciones estimadas bajo estructuras caídas.

Evaluación Crítica y Escepticismo Arquitectónico

Dentro del rigor analítico que requiere una evaluación exhaustiva, debe contemplarse el escepticismo manifestado por foros de ingeniería sobre el rendimiento técnico publicitado del repositorio. En el análisis comunitario recolectado, una fuerte crítica proveniente de un foro de inteligencia artificial calificó la arquitectura técnica de ser "vaporware envuelto en marketing de ciencia ficción". El crítico cuestionó específicamente la disonancia aparente entre un hardware con chip ESP32 de grado pasatiempo, que opera capturando CSI a unos modestos 20\text{ Hz} de actualización, y el documento README.md que publicita métricas extraordinarias como "un pipeline de 54,000 fps (fotogramas por segundo)" equivalente al procesamiento en clústeres empresariales.

No obstante, la evaluación de la base del código, concretamente en la separación lógica del sistema Cliente/Servidor, neutraliza la controversia. El nodo receptor inalámbrico de \\$9 efectivamente empuja un caudal de datos ralo (20 matrices por segundo a través de portadoras multiplexadas). Sin embargo, la tubería analítica principal se delega a la estación de inferencia de alto rendimiento (el motor Rust). La estadística de 54,000 fps (o tiempos de 2.1 segundos para procesar retrospectivamente 400 épocas de un archivo grabado) pertenece a la latencia analítica computacional interna en procesadores acelerados mediante el framework Candle de Hugging Face y GPUs especializadas tipo RTX 5080, permitiendo procesar el entrenamiento temporal masivo a velocidades astronómicas, no el ancho de banda del canal aéreo subyacente.

Una vulnerabilidad funcional real reside en el proceso de calibración espacial continua y transversal. Las redes neuronales que proyectan nubes de puntos 3D asumen y aprenden empíricamente la disposición espacial volumétrica y el rebote estático de la arquitectura específica de cada cuarto. El documento examples/medical denota y asume explícitamente que la precisión inferencial requiere un sujeto estacionario, y la documentación de la matriz de algoritmos señala que cualquier degradación topológica (mover un sillón masivo o resituar físicamente el nodo ESP32 perimetral un palmo hacia la izquierda) colapsa la fiabilidad del modelo de forma catastrófica, requiriendo iniciar obligatoriamente un borrado del estado y la activación de una calibración ambiental forzada.

Plan Exhaustivo de Implementación en Ordenador Portátil y Gestión en la Nube

Para cumplir rigurosamente el requerimiento integral sobre cómo desplegar, implementar, almacenar modelos e instanciar toda la arquitectura RuView sobre una estación de trabajo móvil (portátil) en sincronía permanente con Google Drive, a continuación se desglosa un flujo de instalación paso a paso, abordando las idiosincrasias y correcciones estructurales derivadas de incidencias reportadas en el propio GitHub.

1. Requerimientos Iniciales de Hardware y Software

Un ordenador portátil genérico contemporáneo tiene la capacidad teórica suficiente para ejecutar la visualización y procesamiento nativos, preferiblemente equipado con un sistema operativo de núcleo 64 bits (macOS, Windows 10/11 o distribución Linux soportada), y al menos 4-8 GB de RAM.

El ecosistema debe orquestarse mediante **Docker Desktop** (versiones superiores a la 24.0+ recomendadas) para evitar colapsos complejos de bibliotecas nativas de interfaz gráfica (como librerías de sistema en distribuciones Debian libglib2.0-dev, libgtk-3-dev, o frameworks webkit2gtk), requiriendo simplemente que el motor de virtualización local Hyper-V, WSL2, o Apple Virtualization esté habilitado en el dispositivo host.

Para el componente físico biométrico recomendado (hardware extremo IoT):

- Un microcontrolador comercial estándar **ESP32-S3** (monitor de cuarto WiFi) o un dispositivo avanzado acoplado con el hardware **Seeed MR60BHA2 ESP32-C6** de 60 GHz.
- Presupuesto esperado: \$15 - \$24.
- Cables USB físicos estándar tipo C para proveer datos telemétricos y alimentación al puerto serial COM/ttyUSB.

2. Configuración Dinámica de Modos de Ingesta (CSI_SOURCE)

El proyecto RuView admite versatilidad estructural basada en si el usuario actualmente posee hardware físico con él o no, determinando qué datos analiza la canalización a través de la bandera operativa y de entorno CSI_SOURCE :

- simulated: Modo académico por defecto que sintetiza matemática y virtualmente una matriz de datos espaciales sin periféricos, garantizando el despliegue puramente de testeo.
- wifi: Modo en el que la aplicación Rust consulta el controlador de la red de origen y accede localmente al chipset WiFi integrado del portátil del usuario (por ejemplo, monitorizando el Indicador de Fuerza de Señal Recibida (RSSI) con netsh en Windows o CoreWLAN en macOS OSX). La fiabilidad queda limitada pero extrae la presencia macro. Este modo colisiona fuertemente con la contención del puente de red y carece de soporte orgánico nativo de paso si no se invoca el argumento --network host en contenedores.
- esp32: Modo operacional industrial estándar que abre un servidor de escucha pasiva en el puerto UDP 5005 esperando la matriz binaria asíncrona del hardware perimetral de la sala.

3. Implementación Local con Docker Compose y Enrutamiento Google Drive

Para responder a la necesidad de sincronización perpetua con almacenamiento en red de Google Drive desde el dispositivo portátil, se requiere una integración orquestada utilizando herramientas locales de sincronización. Google Drive dispone de un ejecutable para ordenadores personales ("Drive para escritorio") que monta un disco duro lógico virtual o un espejo de carpetas del sistema local directamente mapeado a la nube remota (ej. C:\Usuarios\Nombre\Mi Unidad\Modelos_RuView en sistemas Windows, o ~/Google Drive/Modelos_RuView en entornos Mac).

La metodología más limpia consiste en construir un perfil topológico en formato docker-compose.yml, que define qué servicios levantar e inyecta la carpeta local sincronizada de Google Drive dentro del sistema de archivos privado del contenedor de forma bidireccional. El usuario deberá crear un archivo docker-compose.yml en su terminal gráfica o editor de código, adhiriéndose al siguiente esqueleto operativo (modificando debidamente la ruta izquierda correspondiente a su directorio Drive validado):

version: "3.9"

services:

sensing-server:

image: ruvnet/wifi-densepose:latest

ports:

Exposición de la API REST para el frontal del usuario

- "3000:3000"

Exposición de los WebSockets de telemetría dinámica en tiempo real

- "3001:3001"

Escucha de tráfico UDP desde el microcontrolador remoto de la habitación

- "5005:5005/udp"

environment:

Definir la escucha específica para el receptor ESP32

- CSI_SOURCE=esp32

Ordenar a la aplicación Rust que busque y grabe modelos neuronales en esta ruta

- MODELS_DIR=/app/models

volumes:

MAPEO ESTRUCTURAL CLAVE PARA SINCRONIZACIÓN EN LA NUBE:

Ruta real del disco local gestionado por Google Drive : Ruta de destino del contenedor interno

- /Ruta/A/Mi_Unidad_Google_Drive/RuView_Models:/app/models

La ejecución del panel local mediante el comando docker-compose up -d instruye a Docker a descargar la imagen arquitectónica preconstruida desde el servidor (docker pull ruvnet/wifi-densepose:latest), instanciando la base de datos interna y los puentes en segundos. Si existieran anomalías documentadas en foros respecto a variables anexadas a flags que devuelven /bin/sh: 0: Illegal option --, este formato YAML soluciona y evita de plano dicha inyección shell errónea.

El servidor es invocado localmente dirigiéndose a <http://localhost:3000> con el navegador web, mostrando el panel volumétrico en tres dimensiones (Three.js) y las analíticas. De igual forma,

las APIs REST asíncronas para el chequeo del estado del nodo son consultables por interfaces JSON como curl `http://localhost:3000/api/v1/sensing/latest` para evaluar presencias macro, y `http://localhost:3000/api/v1/vital-signs` para interrogar biometría cardiorrespiratoria.

4. Flujo Alternativo en Python para Scripts Médicos (Telemetría Directa)

Si el analista está interesado exclusivamente en aislar y ejecutar el modelo clínico de presión arterial mediante el radar mmWave sin levantar todo el orquestador backend de Rust/Docker, el repositorio ofrece dependencias en crudo para lenguajes de scripting dinámicos.

El científico procederá entonces en el terminal del portátil clonando la bifurcación y activando los componentes mínimos desde la carpeta local (no virtualizada):

```
git clone https://github.com/ruvnet/RuView.git
cd RuView
```

```
# Configuración del paquete estándar de matemáticas subyacente
pip install pyserial numpy
```

Conectando la placa USB al portátil y comprobando su enumeración de puerto de placa (supongamos que el OS anfitrión la etiqueta como el puerto COM4), iniciará la lectura serial cruda, introduciendo los datos esfigmomanométricos del sujeto para la correlación tensional de anclaje inicial :

```
python examples/medical/bp_estimator.py --port COM4 \
--cal-systolic 120 --cal-diastolic 80 --cal-hr 72
```

Ecosistema Experimental en la Nube mediante Colaboración Google Drive y Colab

El ciclo vital de las arquitecturas generativas contrastivas no es estático; exige la captación continua de tensores multidimensionales locales y su entrenamiento masivo por propagación hacia atrás. Ejecutar esto localmente en la CPU estándar del portátil tomaría meses computacionales, razón por la que la documentación fomenta el uso de plataformas conjuntas como Google Colab (respaldadas por procesamiento GPU Nvidia A100/T4 en servidores de la nube y sincronizadas a través del almacenamiento montado de Drive).

El flujo bidireccional opera bajo esta coreografía sistémica:

1. **Ingesta de Bases de Datos Multidimensionales Externas:** El sistema puede entrenar directamente de conjuntos de datos estandarizados comunitarios que contienen miles de horas de firmas matriciales espectrales de sujetos monitorizados en cuartos bajo la norma, como las arquitecturas MM-Fi de NeurIPS (formatos NumPy .npy) o Wi-Pose (formatos .mat). Ambos hiperarchivos están pre-alojados comúnmente como repositorios masivos en cuentas gratuitas de Drive que el analista simplemente descargará a la carpeta designada previamente (RuView_Models / MM-Fi_Data).
2. **Captura Activa Asíncrona (Recording):** Utilizando la instancia y el panel local del portátil, el usuario enciende el registro espectral de un escenario determinado (una configuración particular de su habitación local) despachando un comando API REST directo mediante POST `/api/v1/recording/start`. Todo el tráfico del ESP32 capturado y pre-procesado volcará matrices persistentes directamente a la carpeta del ordenador que es administrada activamente por el cliente de Google Drive subyacente, subiendo

automáticamente cada iteración JSONL o de matriz al clúster remoto.

3. **Procesamiento y Compilación Nerviosa Central en Colaboratory:** En la plataforma de Google Colab, se levantará un cuaderno de inferencia Python emparejado mediante la librería google.colab.

```
from google.colab import drive
drive.mount('/content/drive')
```

Adquisición e instalación de los perfiles acelerados GPU para el orquestador

```
!git clone https://github.com/ruvnet/RuView.git %cd RuView !pip install wifi-densepose[all]
wifi-densepose[gpu] ``
```

El entorno en la nube, habiendo accedido a la carpeta montada (donde descansan los datos locales grabados por el portátil), invocará el núcleo compilado del marco de trabajo en Rust (o el puente equivalente de llamadas en Python) para masticar todas las incrustaciones temporales utilizando el pipeline iterativo puro de Rust de 7,832 líneas incorporadas para modelos.

bash # Proceso formativo de modelo mediante épocas en clúster acelerado

```
!./target/release/sensing-se[span_69](start_span)[span_69](end_span)rver --train \ --dataset
/content/drive/MyDrive/MM-Fi_Data/ \ --dataset-type mmfi --epochs 100 \ --save-rvf
/content/drive/MyDrive/RuView_Models/trained_model_v1.rvf
```

1. **Distribución y Retroalimentación Perimetral de Contenedor RVF:** Una vez que el clúster generativo (Google Colab) ha consumido y comprimido matemáticamente los miles de fotogramas, produce y vuelca un encapsulado autónomo en el formato de contenedor binario .rvf (RuVector Format) directo a la partición referenciada de la unidad montada remota. Segundos después, el proceso informático reverso del agente local del portátil del usuario actualizará su clon perimetral detectando la creación del nuevo activo. El analista ejecutará un reinicio dinámico interactivo en caliente con el servidor en su portátil para purgar las estadísticas neuronales heurísticas anteriores e implantar la iteración refinada del marco de peso. Utilizando un único llamado POST a <http://localhost:3000/api/v1/models/load> invocará el recién horneado activo y modificará las constantes del sistema base instantáneamente.

Consideraciones Sistémicas Finales

El ecosistema expuesto en la infraestructura RuView cristaliza un salto exponencial y paradigmático respecto al modelado fisiológico tradicional mediante radiofrecuencia perimetral, subvirtiendo la naturaleza de transceptores masivos y omnipresentes comerciales de grado consumidor (WiFi), integrándolos y fusionándolos algorítmicamente en instrumentación de categoría predictiva espacial superior.

La ingeniería orquestada destila una rigurosa solidez basada en modelaje DDD que fragmenta el dominio acústico, espectral e inteligente en silos protegidos, permitiendo adaptabilidad para que algoritmos de teoría del caos interactúen limpiamente y clasifiquen ambientes variables, al mismo tiempo que delegan análisis respiratorio micrométrico denso a micro-frecuencias dedicadas (60 GHz FMCW). La interoperabilidad metodológica de este sistema facilita su replicación industrial en escenarios críticos inexplorados, tales como protocolos masivos y precisos en desastres de infraestructuras cerradas y entornos de atmósferas agresivas donde metodologías anticuadas exhibirían fallos en cascada instantáneos, todo esto impulsando el avance de un marco perimetral privado que descarta estructuralmente por completo los sesgos e inhibiciones de privacidad óptica endémicas en arquitecturas heredadas del pasado.

Fuentes citadas

1. Cognitum RuView | Privacy-First WiFi Sensing, <https://cognitum.one/RuView> 2. π RuView turns commodity WiFi signals into real-time spatial intelligence, vital sign monitoring, and presence detection — all without a single pixel of video. - GitHub, <https://github.com/ruvnet/RuView> 3. RuView: an open source platform for camera-free spatial sensing with WiFi signals, <https://www.knightli.com/en/2026/05/17/ruview-wifi-sensing-platform/> 4. RuView/examples/medical/README.md at main - GitHub, <https://github.com/ruvnet/RuView/blob/main/examples/medical/README.md> 5. WiFi DensePose User Guide - ruvnet/RuView - GitHub, <https://github.com/ruvnet/RuView/blob/main/docs/user-guide.md> 6. This open-source tool turns any WiFi router into a through-wall body tracking camera, https://www.reddit.com/r/ArtTrailblazers/comments/1robue9/this_opensource_tool_turns_any_wifi_router_into_a/ 7. Contactless blood pressure from a \$15 radar sensor — no wearable, no camera, just physics · Issue #271 · ruvnet/RuView - GitHub, <https://github.com/ruvnet/RuView/issues/271> 8. RuView/README.md at main · ruvnet/RuView · GitHub, <https://github.com/ruvnet/RuView/blob/main/README.md> 9. RuView/.github/workflows/desktop-release.yml at main · ruvnet/RuView · GitHub, <https://github.com/ruvnet/RuView/blob/main/.github/workflows/desktop-release.yml> 10. RuView/docs/build-guide.md at main - GitHub, <https://github.com/ruvnet/RuView/blob/main/docs/build-guide.md> 11. README.md - ruvnet/RuView - GitHub, <https://github.com/ruvnet/RuView/blob/main/docs/ddd/README.md> 12. RuView/docs/ddd/rvcsi-domain-model.md at main - GitHub, <https://github.com/ruvnet/RuView/blob/main/docs/ddd/rvcsi-domain-model.md> 13. RuView/docs/readme-details.md at main - GitHub, <https://github.com/ruvnet/RuView/blob/main/docs/readme-details.md> 14. ruvnet/RuView at toolbrain.net - GitHub, <https://github.com/ruvnet/RuView?ref=toolbrain.net> 15. RuView/docs/edge-modules/adaptive-learning.md at main - GitHub, <https://github.com/ruvnet/RuView/blob/main/docs/edge-modules/adaptive-learning.md> 16. RuView/docs/edge-modules/industrial.md at main - GitHub, <https://github.com/ruvnet/RuView/blob/main/docs/edge-modules/industrial.md> 17. feat: Real-Time Dense Point Cloud — Camera + WiFi CSI + mmWave Sensor Fusion · Issue #406 · ruvnet/RuView - GitHub, <https://github.com/ruvnet/RuView/issues/406> 18. Can't run docker in esp32 mode · ruvnet RuView · Discussion #175 - GitHub, <https://github.com/ruvnet/RuView/discussions/175> 19. Docker latest doesn't include observatory and quite a few other files. · Issue #520 · ruvnet/RuView - GitHub, <https://github.com/ruvnet/RuView/issues/520> 20. Can't run docker in esp32 mode · Issue #153 · ruvnet/RuView - GitHub, <https://github.com/ruvnet/RuView/issues/153> 21. docs: add Docker ESP32 troubleshooting section #214 - GitHub, <https://github.com/ruvnet/RuView/pull/214/files> 22. No, this is not fake. Yes, it actually works. Read the docs. · Issue #37 · ruvnet/RuView - GitHub, <https://github.com/ruvnet/RuView/issues/37>